

Contents

1	LitKit	3
1.1	Key Features	3
1.2	How It Works	4
1.3	Platform Guides	4
1.4	Quick Start	4
1.4.1	Prerequisites	4
1.4.2	Install	4
1.4.3	Build Index	4
1.4.4	Query	4
1.5	Important: Data Preparation	4
1.6	CLI Reference	4
1.6.1	Build Options	4
1.6.2	Query Options	5
1.6.3	Performance Options	5
1.6.4	Other Options	5
1.7	Environment Variables	5
1.8	Documentation	6
1.8.1	Generating PDF Documentation	6
1.9	License	6
2	LitKit: macOS Installation & Usage Guide	6
2.1	Table of Contents	6
2.2	Overview	6
2.2.1	How It Works	7
2.3	System Requirements	7
2.3.1	Hardware	7
2.3.2	Software	7
2.3.3	Install uv	7
2.3.4	Check Your Python Version	7
2.4	Installation	7
2.4.1	Option A: Install from ZIP Archive	7
2.4.2	Option B: Install from Git	8
2.4.3	Verify Installation	8
2.5	Configuring the LLM Backend	8
2.5.1	Option A: Local LLM (Recommended for Privacy)	8
2.5.2	Option B: Online API	9
2.5.3	Listing Available Models	9
2.6	Preparing Your Papers	9
2.6.1	Expected Structure	9
2.6.2	Step 1: Create the Workspace Directory	9
2.6.3	Step 2: Copy Your Papers Archive	10
2.6.4	Step 3: Create a Manifest File	10
2.7	Building the Index	10
2.7.1	Download Embedding Models (Required Once)	10
2.7.2	Build Command (Small Corpus)	10
2.7.3	Build Command (Larger Corpus)	11
2.7.4	Using the Test Script	11
2.7.5	What Happens During Build	11
2.7.6	Expected Output	11
2.7.7	Resume After Interruption	11
2.8	Querying Your Literature	11
2.8.1	Basic Query (Local LLM)	12

2.8.2	Basic Query (Online API)	12
2.8.3	Query with More Context	12
2.8.4	Query from a File	12
2.8.5	Retrieval Only (No LLM)	12
2.8.6	Expected Output Format	12
2.9	Command Reference	12
2.9.1	Build Options	12
2.9.2	Query Options	13
2.9.3	Performance Options	13
2.9.4	Environment Variables	13
2.10	Troubleshooting	14
2.10.1	“No module named ‘litkit’”	14
2.10.2	“Model not found” or Download Errors	14
2.10.3	Connection Refused (Local LLM)	14
2.10.4	“401 Unauthorized” from Online API	14
2.10.5	“No local snapshot found for allenai/specter2_base”	14
2.10.6	“No tar shards found”	14
2.10.7	FAISS Segfaults	15
2.10.8	FAISS Hangs During Build	15
2.10.9	Verifying MPS Setup	15
2.10.10	Index Corruption	15
2.11	Tips & Best Practices	15
2.11.1	1. Start Small	15
2.11.2	2. Use FLAT Indices for Small Corpora	16
2.11.3	3. Use Uncompressed Tar Files for Large Builds	16
2.11.4	4. Save Common Options in a Script	16
2.11.5	5. Backup Your Index	16
2.11.6	6. Check Retrieved Context First	16
2.12	Quick Start Summary	17
2.13	Getting Help	17
3	LitKit: HPC Cluster Guide	17
3.1	Table of Contents	17
3.2	Overview	18
3.2.1	Architecture	18
3.3	Prerequisites	18
3.3.1	Hardware Requirements	18
3.3.2	Software Requirements	18
3.3.3	CUDA Setup Options	18
3.3.4	Container Image	19
3.4	Building the Container	19
3.4.1	Dockerfile Flavors	19
3.4.2	Building with the Justfile	19
3.4.3	Build Commands	20
3.4.4	Regenerating the Lockfile	20
3.5	Caching HuggingFace Models	20
3.5.1	Why Cache Models?	20
3.5.2	Creating a Persistent Cache	20
3.5.3	Converting to SafeTensors	20
3.5.4	Environment Variables for Offline Mode	21
3.6	Understanding Multi-Node Builds	21
3.6.1	When to Use Multi-Node	21
3.6.2	Producer/Consumer Architecture	21
3.6.3	Tar Staging to Local SSD	21

3.7	Why Uncompressed Tar Files?	22
3.7.1	The Problem with Compressed Tars	22
3.7.2	Uncompressed Tar Enables Parallelism	22
3.7.3	Converting Your Archives	22
3.8	Customizing SLURM Batch Scripts	22
3.8.1	Essential Variables	22
3.8.2	Manifest Files	23
3.8.3	SLURM Resource Requests	23
3.9	Running Single-Node Builds	23
3.9.1	When to Use	23
3.9.2	Basic Command	24
3.9.3	Using the Batch Script	24
3.9.4	Key Options	24
3.10	Running Multi-Node Builds	24
3.10.1	When to Use	24
3.10.2	Architecture	24
3.10.3	Using the Batch Script	24
3.10.4	Bootstrap Phase	24
3.10.5	Monitoring Progress	24
3.11	Querying Your Index	25
3.11.1	Using a Question File	25
3.11.2	Interactive Queries	25
3.11.3	With External LLM API	25
3.11.4	Key Query Options	25
3.12	Troubleshooting	25
3.12.1	Verifying GPU Setup	25
3.12.2	Monitoring SLURM Jobs	26
3.12.3	“CUDA not available” inside container	26
3.12.4	“database is locked” errors	26
3.12.5	Low GPU utilization (bursty pattern)	26
3.12.6	Producer failed, consumer waiting forever	27
3.12.7	Stale writer guard file	27
3.12.8	“pam_slurm_adapt” error when SSH to node	27
3.13	Command Reference	27
3.13.1	Environment Variables	27
3.13.2	Key CLI Flags	28
3.13.3	Utility Scripts	28

1 LitKit

A scalable two-stage RAG (Retrieval-Augmented Generation) pipeline for scientific literature. Build searchable vector indices from JATS/NXML XML corpora and query with natural language questions.

1.1 Key Features

- **Two-stage retrieval** — SPECTER2 for paper-level search, SBERT for passage-level search
- **Streaming tar ingestion** — process tar archives without extraction
- **Resumable builds** — checkpoint-based recovery from interruption
- **Multi-node support** — producer/consumer architecture for HPC clusters
- **Air-gapped operation** — works offline with cached HuggingFace models
- **Token-budgeted prompts** — automatic context trimming for LLM limits

1.2 How It Works

LitKit uses a **two-stage retrieval** approach:

1. **Stage 1 (Papers):** SPECTER2 embeddings index titles and abstracts for fast paper-level similarity search
2. **Stage 2 (Chunks):** SBERT embeddings index body text chunks for passage-level retrieval
3. **LLM Synthesis:** Top chunks are passed to an LLM which generates an answer with citations

1.3 Platform Guides

Platform	Guide	Description
HPC Cluster	LITKIT_CLUSTER_GUIDE.md	HPC deployment with Charliecloud, multi-node builds, GPU passthrough
macOS	LITKIT_MAC_GUI_DE.md	Local development on MacBook (Apple Silicon or Intel)

1.4 Quick Start

1.4.1 Prerequisites

- **Python 3.12** (exact version required)
- **uv** package manager

```
# Install uv if needed
curl -LsSf https://astral.sh/uv/install.sh | sh
```

1.4.2 Install

```
git clone https://github.com/lanl/litkit.git
cd litkit
uv sync
```

1.4.3 Build Index

```
# Place tar archives in workspace/tar_shards/
litkit --build-only --faiss-writer --tar-dir workspace/tar_shards
```

1.4.4 Query

```
litkit "What mechanisms are described in these papers?"
```

1.5 Important: Data Preparation

LitKit does not deduplicate papers. Each XML file in your tar archives is treated as a unique document.

- **Your responsibility:** Ensure tar files do not contain duplicate XML files
- **If duplicates exist:** The same paper will be indexed multiple times
- **Recommended:** Use uncompressed **.tar** files (not **.tar.gz**) for parallel processing

1.6 CLI Reference

1.6.1 Build Options

Option	Description
<code>--build-only</code>	Build indices without running a query
<code>--faiss-writer</code>	Enable FAISS index writes (required for builds)
<code>--rebuild</code>	Wipe and rebuild from scratch
<code>--update</code>	Append new files only
<code>--tar-dir DIR</code>	Directory containing tar shards
<code>--tar-manifest FILE</code>	File listing tar paths (one per line)

1.6.2 Query Options

Option	Description
<code>--llm-model MODEL</code>	LLM model name (default: <code>gpt-oss:20b</code>)
<code>--top-papers N</code>	Stage 1 shortlist size (default: 500)
<code>--top-chunks N</code>	Chunks for LLM context (default: 30)
<code>--no-llm</code>	Retrieval only, skip LLM call
<code>--per-paper-cap N</code>	Max chunks per paper in context (default: 3)

1.6.3 Performance Options

Option	Description
<code>--embed-devices SPEC</code>	Device selection: <code>auto</code> , <code>cpu</code> , <code>mps</code> , <code>cuda:0</code> , <code>cuda:1</code>
<code>--paper-embed-bs N</code>	Paper embedding batch size (default: 16)
<code>--chunk-embed-bs N</code>	Chunk embedding batch size (default: 64)
<code>--parse-workers N</code>	Parallel XML parsing threads (default: 8)

1.6.4 Other Options

Option	Description
<code>--quiet</code>	Suppress progress output
<code>-y</code> , <code>--yes</code>	Skip confirmation prompts
<code>--offline</code>	Force HuggingFace offline mode
<code>--version</code>	Print version and exit

For the complete list, run `litkit --help`.

1.7 Environment Variables

Variable	Description
<code>LITKIT_WORKSPACE</code>	Output directory for indices and database
<code>LITKIT_TAR_DIR</code>	Default tar shard directory
<code>HF_HOME</code>	HuggingFace model cache location
<code>HF_HUB_OFFLINE=1</code>	Force offline mode
<code>LITKIT_QUIET=1</code>	Suppress output (like <code>--quiet</code>)
<code>LITKIT_DEBUG=1</code>	Enable verbose debugging

1.8 Documentation

- **Platform Guides:** See LITKIT_CLUSTER_GUIDE.md and LITKIT_MAC_GUIDE.md
- **Developer Guide:** See docs/DEVELOPER_GUIDE.md for internals, optimization strategies, and version bump procedures

1.8.1 Generating PDF Documentation

```
# Install prerequisites (macOS)
brew install pandoc
brew install --cask basictex

# Generate combined PDF
pandoc README.md LITKIT_MAC_GUIDE.md LITKIT_CLUSTER_GUIDE.md \
  --pdf-engine=xelatex \
  --toc --number-sections \
  -V geometry:margin=1in \
  -o litkit_documentation.pdf
```

1.9 License

LitKit is released under the MIT License; see LICENSE. LANL software release O5068.

© 2026. Triad National Security, LLC. All rights reserved. This program was produced under U.S. Government contract 89233218CNA000001 for Los Alamos National Laboratory (LANL), which is operated by Triad National Security, LLC for the U.S. Department of Energy/National Nuclear Security Administration. All rights in the program are reserved by Triad National Security, LLC, and the U.S. Department of Energy/National Nuclear Security Administration. The Government is granted for itself and others acting on its behalf a nonexclusive, paid-up, irrevocable worldwide license in this material to reproduce, prepare derivative works, distribute copies to the public, perform publicly and display publicly, and to permit others to do so.

2 LitKit: macOS Installation & Usage Guide

A step-by-step guide for installing and using LitKit on macOS to query scientific literature.

2.1 Table of Contents

1. Overview
 2. System Requirements
 3. Installation
 4. Configuring the LLM Backend
 5. Preparing Your Papers
 6. Building the Index
 7. Querying Your Literature
 8. Command Reference
 9. Troubleshooting
 10. Tips & Best Practices
-

2.2 Overview

LitKit is an air-gapped RAG (Retrieval-Augmented Generation) pipeline designed for scientific literature search. It enables you to:

- **Ingest** scientific papers from JATS/NXML format (commonly used by PubMed Central)
- **Embed** paper titles, abstracts, and body text using state-of-the-art models
- **Index** embeddings for fast similarity search using FAISS
- **Query** your literature collection with natural language questions
- **Answer** using an LLM with citations to the source material

2.2.1 How It Works

LitKit uses a **two-stage retrieval** approach:

1. **Stage 1 (Papers):** Your question is embedded using SPECTER2 (a scientific paper embedding model) and matched against paper-level embeddings to find the most relevant papers.
2. **Stage 2 (Chunks):** Text chunks from the shortlisted papers are searched using SBERT embeddings to find the most relevant passages.
3. **LLM Synthesis:** The top chunks are passed to an LLM which synthesizes an answer with citations.

2.3 System Requirements

2.3.1 Hardware

- **macOS:** Sonoma, Ventura, or Monterey (Apple Silicon or Intel)
- **RAM:** 8 GB minimum, 16 GB recommended
- **Disk:** ~5 GB for models + index storage (varies with corpus size)

2.3.2 Software

- **Python 3.12** (exact version required)
- **uv** (Python package manager) — recommended

2.3.3 Install uv

```
# Install uv (if not already installed)
curl -LsSf https://astral.sh/uv/install.sh | sh
```

```
# Verify installation
uv --version
```

2.3.4 Check Your Python Version

```
python3 --version
# Should output: Python 3.12.x
```

If you don't have Python 3.12, install it via Homebrew:

```
brew install python@3.12
```

2.4 Installation

2.4.1 Option A: Install from ZIP Archive

If you received LitKit as a ZIP file:

```
# 1. Extract the archive
unzip litkit-v0.3.35.zip
```

```
cd litkit

# 2. Install all dependencies (uv.lock ensures correct versions)
uv sync

# 3. Verify installation
uv run litkit --version
```

2.4.2 Option B: Install from Git

```
# 1. Clone the repository
git clone https://github.com/lanl/litkit.git
cd litkit

# 2. Install all dependencies (uv.lock ensures correct versions)
uv sync

# 3. Verify installation
uv run litkit --version
```

2.4.3 Verify Installation

```
uv run litkit --version
# Should output: litkit 0.3.35

# Verify MPS is available (Apple Silicon)
uv run python -c "import torch; print('MPS:', torch.backends.mps.is_available())"

# Verify FAISS
uv run python -c "import faiss; print('FAISS OK')"
```

```
# Verify device detection
uv run python -c "from litkit.embeddings.devices import detect_device; print('Device:', detect_device())"
```

2.5 Configuring the LLM Backend

LitKit uses an OpenAI-compatible API to generate answers. You can use either a **local LLM** (LM Studio, Ollama) or an **online API** (OpenAI, Anthropic, etc.).

2.5.1 Option A: Local LLM (Recommended for Privacy)

Run an LLM locally using LM Studio or Ollama:

LM Studio (default port 1234):

```
# No API key needed for local models
litkit "What is the main finding?" \
    --llm-model lmstudio-community/Meta-Llama-3.1-8B-Instruct-GGUF \
    --openai-base-url "http://localhost:1234/v1"
```

Ollama (default port 11434):

```
litkit "What is the main finding?" \
    --llm-model llama3.2 \
    --openai-base-url "http://localhost:11434/v1"
```

Advantages: - Complete privacy—data never leaves your machine - No API costs - Works offline

2.5.2 Option B: Online API

Use OpenAI, Anthropic, or any OpenAI-compatible API:

```
# Store your API key in a file (not an environment variable)
echo "YOUR_API_KEY" > ~/.llm_api_key
chmod 600 ~/.llm_api_key

# Query with OpenAI
litkit "What is the main finding?" \
  --llm-model gpt-4 \
  --openai-base-url "https://api.openai.com/v1" \
  --openai-api-key "$(cat ~/.llm_api_key)"
```

Security Note: Store your API key in a file rather than an environment variable. Environment variables set via `export` may appear in shell history files and process listings. Using `$(cat ~/.llm_api_key)` reads the key at runtime without exposing it in logs or history.

2.5.3 Listing Available Models

Query any OpenAI-compatible endpoint to see available models:

```
# LM Studio (local, no auth)
curl -s http://localhost:1234/v1/models | jq '.data[].id'

# Ollama
curl -s http://localhost:11434/v1/models | jq '.data[].id'

# OpenAI (with auth)
curl -s -H "Authorization: Bearer $(cat ~/.llm_api_key)" \
  https://api.openai.com/v1/models | jq '.data[].id'
```

2.6 Preparing Your Papers

LitKit ingests papers from `.tar` or `.tar.gz` archives containing JATS/NXML XML files. If your papers were converted from PDFs using the `text-fetch` utility, they should already be in the correct format.

2.6.1 Expected Structure

Your tar archive should contain XML files at the top level or in subdirectories:

```
papers.tar
  paper1.xml
  paper2.xml
  paper3.xml
  ...
```

Note: For larger builds, uncompressed `.tar` files are faster than `.tar.gz` because they allow parallel XML parsing. See the performance tips section.

2.6.2 Step 1: Create the Workspace Directory

```
# From the litkit directory
mkdir -p workspace/tar_shards
```

2.6.3 Step 2: Copy Your Papers Archive

```
cp /path/to/your/papers.tar workspace/tar_shards/
```

2.6.4 Step 3: Create a Manifest File

The manifest tells LitKit which archives to process:

```
echo "papers.tar" > workspace/papers.manifest
```

Or if you have multiple archives:

```
cat > workspace/papers.manifest << 'EOF'
papers_batch1.tar
papers_batch2.tar
papers_batch3.tar
EOF
```

2.7 Building the Index

Before you can query your papers, LitKit needs to build a searchable index.

2.7.1 Download Embedding Models (Required Once)

LitKit requires two embedding models from HuggingFace (~1 GB total). **You must download them before your first build:**

```
# Set cache location
export HF_HOME=$(pwd)/workspace/hf_cache

# Download models
uv run python -c "
from transformers import AutoModel, AutoTokenizer
from sentence_transformers import SentenceTransformer

print('Downloading SPECTER2...')
AutoTokenizer.from_pretrained('allenai/specter2_base')
AutoModel.from_pretrained('allenai/specter2_base')

print('Downloading SBERT...')
SentenceTransformer('sentence-transformers/all-mpnet-base-v2')

print('Done!')
"
```

Note: Models are cached in `workspace/hf_cache/` and only need to be downloaded once.

2.7.2 Build Command (Small Corpus)

For a small collection of papers (< 10,000), use FLAT indices for simplicity:

```
uv run litkit --build-only \
  --faiss-writer \
  --tar-dir workspace/tar_shards \
  --papers-index flat \
  --chunks-index flat
```

Or using a manifest file:

```
uv run litkit --build-only \  
    --faiss-writer \  
    --tar-manifest workspace/papers.manifest \  
    --papers-index flat \  
    --chunks-index flat
```

2.7.3 Build Command (Larger Corpus)

For larger collections (> 10,000 papers), use HNSW for papers and IVF-PQ for chunks:

```
uv run litkit --build-only \  
    --faiss-writer \  
    --tar-manifest workspace/papers.manifest \  
    --papers-index hnsw \  
    --chunks-index ivfpq
```

2.7.4 Using the Test Script

For development and testing, use `test_build.sh`:

```
# Clean build with default test data
```

```
./test_build.sh --clean
```

```
# With verbose output
```

```
./test_build.sh --clean --verbose
```

2.7.5 What Happens During Build

1. **Scanning:** LitKit streams through your tar archives without extracting
2. **Parsing:** XML files are parsed to extract title, abstract, and body paragraphs
3. **Chunking:** Body text is split into ~1200-character chunks with overlap
4. **Embedding:** Text is embedded using the downloaded models
5. **Indexing:** Embeddings are added to FAISS indices
6. **Saving:** SQLite database and FAISS indices are written to `workspace/`

2.7.6 Expected Output

```
[version] litkit 0.3.35  
[device] using mps                # or cpu on Intel Macs  
[paths] using workspace/tar_shards as source directory for tar shards  
[paths] using /path/to/litkit/workspace as writable directory  
[build] using DB at workspace/sqlite/litkit.sqlite3  
[scan] found 1 tar shards in shard 0/1  
[progress] [scan] papers.tar: 50/50 (100.0%) 12.3/s  
[done] indexed 50 papers and 847 chunks (this run)
```

2.7.7 Resume After Interruption

If the build is interrupted, simply re-run the same command. LitKit maintains checkpoints and will resume from where it left off.

2.8 Querying Your Literature

Once the index is built, you can ask questions about your papers.

2.8.1 Basic Query (Local LLM)

```
litkit "What is the main mechanism described in these papers?" \
  --llm-model llama3.2 \
  --openai-base-url "http://localhost:1234/v1"
```

2.8.2 Basic Query (Online API)

```
litkit "What is the main mechanism described in these papers?" \
  --llm-model gpt-4 \
  --openai-base-url "https://api.openai.com/v1" \
  --openai-api-key "$(cat ~/.llm_api_key)"
```

2.8.3 Query with More Context

Retrieve more chunks for a more comprehensive answer:

```
litkit "What experimental methods were used?" \
  --llm-model llama3.2 \
  --openai-base-url "http://localhost:1234/v1" \
  --top-papers 100 \
  --top-chunks 30
```

2.8.4 Query from a File

For longer questions, save them to a file:

```
echo "What are the key findings regarding the relationship between X and Y?" > question.txt
```

```
litkit --question-file question.txt \
  --llm-model llama3.2 \
  --openai-base-url "http://localhost:1234/v1"
```

2.8.5 Retrieval Only (No LLM)

To see what chunks would be selected without calling the LLM:

```
litkit "What is HIV?" --no-llm
```

This prints the retrieved context, useful for debugging or when you want to manually review sources.

2.8.6 Expected Output Format

Based on the provided context, the main mechanism involves... [1, 2].

The experimental results demonstrate that... [3].

REFERENCES

=====

- [1] Smith et al. "Title of Paper One" (PMID:12345678)
- [2] Jones et al. "Title of Paper Two" (PMCID:PMC9876543)
- [3] Wang et al. "Title of Paper Three" (PMID:11111111)

2.9 Command Reference

2.9.1 Build Options

Flag	Description	Default
--build-only	Build indices without running a query	-
--faiss-writer	Enable index modifications (required for builds)	-
--tar-dir PATH	Directory containing tar archives	workspace/tar_shards
--tar-manifest FILE	Text file listing tar archives (one per line)	-
--papers-index {hns,flat}	Index type for papers	hns
--chunks-index {ivfpq,flat}	Index type for chunks	ivfpq
--rebuild	Wipe existing index and rebuild from scratch	-
--update	Add new files without reprocessing existing ones	-
--parse-workers N	Number of parallel XML parsing threads	8

2.9.2 Query Options

Flag	Description	Default
--llm-model MODEL	LLM model name	-
--openai-base-url URL	API endpoint URL	localhost:1234
--openai-api-key KEY	API authentication key	-
--top-papers N	Number of candidate papers (Stage 1)	500
--top-chunks N	Number of chunks for LLM context (Stage 2)	30
--no-llm	Retrieval only; skip LLM call	-
--question-file FILE	Read question from file (use - for stdin)	-

2.9.3 Performance Options

Flag	Description	Default
--embed-devices DEVS	Embedding devices (auto, cpu, mps, cuda:0)	auto
--paper-embed-bs N	Batch size for paper embeddings	16
--chunk-embed-bs N	Batch size for chunk embeddings	64
--per-paper-cap N	Max chunks per paper in final context	3

2.9.4 Environment Variables

Variable	Description
LITKIT_WORKSPACE	Override workspace directory
HF_HOME	HuggingFace cache location
HF_HUB_OFFLINE=1	Force offline mode (no model downloads)
FAISS_NUM_THREADS=1	Force single-threaded FAISS (fixes hangs)
LITKIT_DEBUG=1	Enable verbose debug output

2.10 Troubleshooting

2.10.1 “No module named ‘litkit’”

Make sure you activated the virtual environment:

```
source .venv/bin/activate
```

2.10.2 “Model not found” or Download Errors

LitKit downloads models on first use. Ensure you have internet access. If behind a proxy:

```
export HTTPS_PROXY=http://proxy.example.com:8080
export HTTP_PROXY=http://proxy.example.com:8080
```

2.10.3 Connection Refused (Local LLM)

Make sure your local LLM server is running:

```
# Check if LM Studio is listening
curl http://localhost:1234/v1/models

# Check if Ollama is running
curl http://localhost:11434/v1/models
```

2.10.4 “401 Unauthorized” from Online API

Check that your API key is correct:

```
cat ~/.llm_api_key
```

Test with curl:

```
curl -H "Authorization: Bearer $(cat ~/.llm_api_key)" \
https://api.openai.com/v1/models
```

2.10.5 “No local snapshot found for allenai/specter2_base”

The embedding models haven’t been downloaded yet. Run the model download step from the “Building the Index” section:

```
export HF_HOME=$(pwd)/workspace/hf_cache
uv run python -c "
from transformers import AutoModel, AutoTokenizer
from sentence_transformers import SentenceTransformer
AutoTokenizer.from_pretrained('allenai/specter2_base')
AutoModel.from_pretrained('allenai/specter2_base')
SentenceTransformer('sentence-transformers/all-mpnet-base-v2')
print('Done!')
"
```

2.10.6 “No tar shards found”

Ensure your tar file is in the correct location and has a valid extension:

```
ls workspace/tar_shards/
# Should show: papers.tar or papers.tar.gz
```

2.10.7 FAISS Segfaults

FAISS can segfault if dependencies are mismatched. Reinstall in a fresh environment:

```
# Remove old environment and reinstall
rm -rf .venv
uv sync
```

2.10.8 FAISS Hangs During Build

FAISS can hang on some systems. Set single-threaded mode:

```
export FAISS_NUM_THREADS=1
litkit --build-only --faiss-writer ...
```

The `test_build.sh` script sets this automatically.

2.10.9 Verifying MPS Setup

On Apple Silicon Macs, LitKit should automatically use MPS (Metal Performance Shaders):

```
# Check MPS availability
uv run python -c "import torch; print('MPS:', torch.backends.mps.is_available())"
# Should output: MPS: True

# Check device detection
uv run python -c "from litkit.embeddings.devices import detect_device; print('Device:', detect_device())"
# Should output: Device: mps
```

If MPS shows False but you have Apple Silicon, try reinstalling:

```
rm -rf .venv
uv sync
```

2.10.10 Index Corruption

If you suspect index corruption, rebuild from scratch:

```
litkit --rebuild --faiss-writer \
      --tar-manifest workspace/papers.manifest \
      --papers-index flat \
      --chunks-index flat -y
```

The `-y` flag skips the confirmation prompt.

2.11 Tips & Best Practices

2.11.1 1. Start Small

For testing, create a manifest with just one or two papers first:

```
# Single paper test
litkit --build-only --faiss-writer \
      --tar-dir workspace/tar_shards \
      --papers-index flat --chunks-index flat
```

2.11.2 2. Use FLAT Indices for Small Corpora

For fewer than ~10,000 papers, FLAT indices are simpler and often faster than HNSW/IVF-PQ:

```
--papers-index flat --chunks-index flat
```

2.11.3 3. Use Uncompressed Tar Files for Large Builds

For builds with many papers, uncompressed .tar files allow parallel XML parsing:

```
# Convert compressed to uncompressed  
gunzip -k papers.tar.gz # Creates papers.tar, keeps original
```

Then use --parse-workers to control parallelism:

```
litkit --build-only --faiss-writer --parse-workers 8 ...
```

2.11.4 4. Save Common Options in a Script

Create a helper script for repeated use:

For local LLM:

```
cat > query.sh << 'EOF'  
#!/bin/bash  
source .venv/bin/activate  
litkit "$1" \  
    --llm-model llama3.2 \  
    --openai-base-url "http://localhost:1234/v1"  
EOF  
chmod +x query.sh
```

```
# Usage:  
./query.sh "What is the main finding?"
```

For online API:

```
cat > query-online.sh << 'EOF'  
#!/bin/bash  
source .venv/bin/activate  
litkit "$1" \  
    --llm-model gpt-4 \  
    --openai-base-url "https://api.openai.com/v1" \  
    --openai-api-key "$(cat ~/.llm_api_key)"  
EOF  
chmod +x query-online.sh
```

2.11.5 5. Backup Your Index

The built index is stored in workspace/. Back it up to avoid rebuilding:

```
tar czf litkit-workspace-backup.tar.gz workspace/sqlite workspace/indices
```

2.11.6 6. Check Retrieved Context First

Before relying on LLM answers, use --no-llm to verify the right papers/chunks are being retrieved.

2.12 Quick Start Summary

```
# 1. Extract and install
unzip litkit-v0.3.35.zip && cd litkit
uv sync

# 2. Download embedding models (once)
export HF_HOME=$(pwd)/workspace/hf_cache
uv run python -c "
from transformers import AutoModel, AutoTokenizer
from sentence_transformers import SentenceTransformer
AutoTokenizer.from_pretrained('allenai/specter2_base')
AutoModel.from_pretrained('allenai/specter2_base')
SentenceTransformer('sentence-transformers/all-mpnet-base-v2')
print('Done!')
"

# 3. Add your papers
mkdir -p workspace/tar_shards
cp /path/to/papers.tar workspace/tar_shards/

# 4. Build index
uv run litkit --build-only --faiss-writer \
    --tar-dir workspace/tar_shards \
    --papers-index flat --chunks-index flat

# 5. Query (local LLM)
uv run litkit "What is the main finding?" \
    --llm-model llama3.2 \
    --openai-base-url "http://localhost:1234/v1"
```

2.13 Getting Help

- **Version:** `litkit --version`
 - **Full help:** `litkit --help`
 - **Author:** William S. Hlavacek (hlavacek@lanl.gov)
 - **Issues:** <https://github.com/lanl/litkit/issues>
-

LitKit v0.3.35 — Air-gapped RAG for Scientific Literature

3 LitKit: HPC Cluster Guide

A guide for running LitKit on HPC clusters using Charliecloud containers with GPU passthrough.

3.1 Table of Contents

1. Overview
2. Prerequisites
3. Building the Container
4. Caching HuggingFace Models
5. Understanding Multi-Node Builds
6. Why Uncompressed Tar Files?

7. Customizing SLURM Batch Scripts
8. Running Single-Node Builds
9. Running Multi-Node Builds
10. Querying Your Index
11. Troubleshooting
12. Command Reference

3.2 Overview

LitKit processes scientific literature corpora (JATS/NXML XML files in tar archives) into searchable vector indices for RAG (Retrieval-Augmented Generation) pipelines.

On HPC clusters, LitKit runs inside a Charliecloud container with GPU passthrough via NVIDIA CDI (Container Device Interface). This guide covers:

- **Single-node builds:** For small-to-medium corpora (< 50 GB)
- **Multi-node builds:** For large corpora (50+ GB) using producer/consumer architecture
- **Querying:** Natural language questions against your indexed literature

3.2.1 Architecture

[Tar Archives] → [LitKit Build] → [FAISS Indices + SQLite DB] → [Query]

Single-node (one process)	Multi-node (producers + consumer)
------------------------------	--------------------------------------

3.3 Prerequisites

3.3.1 Hardware Requirements

Resource	Minimum	Recommended
GPU	1× CUDA-capable	2+ GPUs per node
GPU Memory	16 GB	32+ GB
System Memory	64 GB	128+ GB
Local SSD	100 GB	500+ GB (for staging)

3.3.2 Software Requirements

- **Charliecloud** 0.37+ with CDI support
- **CUDA Toolkit** 12.x (for GPU library bindings)
- **NVIDIA CDI specs** generated for your GPU nodes
- **uv** package manager (for lockfile regeneration)

3.3.3 CUDA Setup Options

You need CUDA libraries available on the host to bind into the container. Two approaches:

Option A: Use site CUDA module (recommended on managed clusters)

```
module load cuda/12.5.0
export CUDA_HOME="${CUDA_HOME:-$(dirname "$(dirname "$(which nvcc)")")}"
```

```
# Pick ARM runtime lib directory (SBSA for Grace/Hopper, aarch64 for older)
if [ -d "$CUDA_HOME/targets/sbsa-linux/lib" ]; then
    export CUDA_LIBA="$CUDA_HOME/targets/sbsa-linux/lib"
elif [ -d "$CUDA_HOME/targets/aarch64-linux/lib" ]; then
    export CUDA_LIBA="$CUDA_HOME/targets/aarch64-linux/lib"
fi
export CUDA_LIBB="$CUDA_HOME/lib64"
```

Option B: Extract from NVIDIA container (for clusters without CUDA modules)

```
module load charliecloud
ch-image pull nvidia/cuda:12.5.0-devel-ubuntu22.04

DEST=/path/to/cuda-12.5-host
mkdir -p "$DEST"
ch-run nvidia/cuda:12.5.0-devel-ubuntu22.04 -- bash -lc \
    'tar -C /usr/local -cf - cuda-12.5' | tar -C "$DEST" -xvf -

export CUDA_BASE="$DEST/cuda-12.5"
export CUDA_LIBA="$CUDA_BASE/targets/sbsa-linux/lib"
export CUDA_LIBB="$CUDA_BASE/lib64"
```

3.3.4 Container Image

LitKit is distributed as a SquashFS container image:

```
litkit-v0.3.35-aarch64-lean.sqfs    # ARM64 (aarch64)
litkit-v0.3.35-x86_64-lean.sqfs    # x86-64 (if available)
```

3.4 Building the Container

LitKit containers are built using the repo's `justfile` and either `Dockerfile.lean` or `Dockerfile.fat`.

3.4.1 Dockerfile Flavors

Flavor	File	CUDA in Image	Portability	Use Case
lean	<code>Dockerfile.lean</code>	No	High	Recommended for most clusters
fat	<code>Dockerfile.fat</code>	Yes	Low	Single-cluster deployments

lean (recommended): No CUDA userspace baked in. At runtime, you bind host CUDA libraries via CDI. More portable—works across clusters with different driver versions.

fat: CUDA userspace injected at build time. Simpler to run, but baked libs must be site driver version.

3.4.2 Building with the Justfile

```
# Ensure Charliecloud is loaded
module load charliecloud

# Build lean container (default)
just build
```

```
# Output: sqfs/litkit-v0.3.35-<arch>-lean.sqfs
```

3.4.3 Build Commands

Command	Description
<code>just build</code>	Build container and export to sqfs
<code>just release</code>	Build + save wheel + export (full release)
<code>just reset</code>	Clear Charliecloud build cache

3.4.4 Regenerating the Lockfile

If you modify `pyproject.toml` (add/remove dependencies), regenerate `uv.lock`:

```
# Install uv if needed
pip install uv
```

```
# Regenerate lockfile
uv lock --python 3.12
```

The lockfile ensures reproducible builds.

3.5 Caching HuggingFace Models

LitKit uses two embedding models from HuggingFace: - **SPECTER2** (`allenai/specter2_base`) for paper-level embeddings - **SBERT** (`sentence-transformers/all-mpnet-base-v2`) for chunk-level embeddings

3.5.1 Why Cache Models?

Air-gapped HPC clusters cannot download models at runtime. You must: 1. Pre-download models to a persistent cache directory 2. Bind the cache into the container 3. Set `HF_HUB_OFFLINE=1` to prevent download attempts

3.5.2 Creating a Persistent Cache

```
# Create cache directory (shared filesystem, persistent across jobs)
mkdir -p /path/to/hf_cache
export HF_HOST=/path/to/hf_cache
```

3.5.3 Converting to SafeTensors

PyTorch's default model format (`.bin`) uses pickle, which has security concerns and slower loading. **SafeTensors** is preferred:

- No arbitrary code execution (safer)
- Memory-mapped loading (faster)
- Required by LitKit when `TRANSFORMERS_USE_SAFE_TENSORS=1`

The `setup_safetensors.sh` script handles conversion:

```
# Set required variables
export IMG=/path/to/litkit.sqfs
export HF_HOST=/path/to/hf_cache
export CUDA_BASE=/path/to/cuda-12.x
export CUDA_LIBA="$CUDA_BASE/targets/sbsa-linux/lib"
```

```
export CUDA_LIBB="$CUDA_BASE/lib64"

# Optional: CDI spec dir (if you have GPU access)
export CDI_SPEC_DIR=/path/to/cdi-specs

# Run conversion (idempotent - safe to run multiple times)
./setup_safetensors.sh
```

The script: 1. Seeds the host cache from the container image 2. Converts `.bin` → `.safetensors` for SPECTER2 and SBERT 3. Removes `.bin` artifacts after successful conversion

3.5.4 Environment Variables for Offline Mode

```
--set-env="HF_HOME=/app/hf_cache"
--set-env="HF_HUB_OFFLINE=1"
--set-env="TRANSFORMERS_OFFLINE=1"
--set-env="TRANSFORMERS_USE_SAFE_TENSORS=1"
```

3.6 Understanding Multi-Node Builds

3.6.1 When to Use Multi-Node

Corpus Size	Recommended Mode
< 10 GB	Single-node
10-50 GB	Single-node (or 2-node)
50-200 GB	3-4 nodes
200+ GB	4+ nodes

3.6.2 Producer/Consumer Architecture

For large corpora, LitKit uses a producer/consumer model:

```
[Producer 0]
[Producer 1]      segments/   [Consumer/Writer]   FAISS indices
[Producer N]
```

- **Producers** (`--embed-producer`): Scan tar files, embed text, write segment files
- **Consumer** (`--consume-only --faiss-writer`): Ingest segments into FAISS indices

CRITICAL: Node Count Must Be Consistent

Once you start a multi-node build with N producer nodes, you **MUST** restart with exactly N producer nodes. Changing the node count mid-build will corrupt your vector store.

Why? Each producer writes to shard-specific files: - SQLite: `litkit_shard_00.sqlite3`, `litkit_shard_01.sqlite3`, ... - Segments: `paper_seg_shard00_*.npz`, `chunk_seg_shard01_*.npz`, ... - Checkpoints: `ckpt_shard_0.json`, `ckpt_shard_1.json`, ...

The consumer expects exactly N shards. Adding or removing nodes breaks this mapping.

3.6.3 Tar Staging to Local SSD

The multi-node batch script (`vector_build_multi.sbatch`) implements **local staging** to avoid NFS bottlenecks:

Problem: Writing many small files to NFS/Lustre causes “metadata storms” that slow down all cluster users.

Solution: Producers write to node-local SSD, then rsync to NFS at completion.

Producer workflow:

1. Write segments to /local/ssd/litkit_jobid_shard0/
2. Write SQLite to /local/ssd/litkit_jobid_shard0/
3. On success: rsync everything to NFS
4. Cleanup local staging directory

Benefits: - 47× faster for small-file creates (vs direct NFS writes) - No NFS lock contention between producers - Failed producers leave local artifacts for debugging

3.7 Why Uncompressed Tar Files?

LitKit strongly recommends using uncompressed `.tar` files instead of `.tar.gz`.

3.7.1 The Problem with Compressed Tars

`.tar.gz` decomposition: [gz stream] → [single thread] → [tar entries]

Cannot parallelize!

Gzip decomposition is inherently **serial**—you cannot seek to random offsets in a compressed stream. This means: - Only one CPU can decompress at a time - XML parsing must wait for decompression - GPUs sit idle during I/O

3.7.2 Uncompressed Tar Enables Parallelism

`.tar` file: [tar entries] → [parallel workers] → [parsed XML]

Random access lxml releases GIL

With uncompressed tars: - Multiple workers can seek to different tar members simultaneously - lxml releases the Python GIL during parsing - The `--parse-workers` flag controls parallelism (default: 8)

3.7.3 Converting Your Archives

```
# Decompress (keeps original)
gunzip -k corpus.tar.gz
```

```
# Or create uncompressed copy
zcat corpus.tar.gz > corpus.tar
```

Trade-off: Uncompressed files are 3-5× larger, but parsing is 5-10× faster on multi-core systems.

3.8 Customizing SLURM Batch Scripts

The batch scripts (`vector_build_single.sbatch`, `vector_build_multi.sbatch`) require customization for your cluster. Key variables to modify:

3.8.1 Essential Variables

```
# CUSTOMIZE FOR YOUR CLUSTER:

# Where your LitKit installation lives
export LITKIT_REPO="/path/to/litkit"
```

```

# Shared filesystem workspace (NFS/Lustre)
export LITKIT_WORKSPACE="${LITKIT_REPO}/workspace"

# Node-local fast storage (SSD, NVMe, or tmpfs)
# Used for staging to avoid NFS bottlenecks
export LOCAL_SSD="/local/scratch"

# CDI spec directory for GPU passthrough
export CDI_SPEC_DIR="/path/to/cdi-specs"

# CUDA toolkit libraries
export CUDA_BASE="/path/to/cuda-12.x"
export CUDA_LIBA="${CUDA_BASE}/targets/sbsa-linux/lib"    # ARM64
export CUDA_LIBB="${CUDA_BASE}/lib64"

# HuggingFace model cache (persistent across jobs)
export HF_HOST="/path/to/hf_cache"

# Container image path
export IMG="${LITKIT_REPO}/sqfs/litkit-v0.3.35-aarch64-lean.sqfs"

```

3.8.2 Manifest Files

Create a manifest file listing tar archive paths (one per line):

```

cat > workspace/corpus.manifest << 'EOF'
/data/corpus/part_001.tar
/data/corpus/part_002.tar
/data/corpus/part_003.tar
EOF

```

Set in batch script:

```

export LITKIT_TAR_MANIFEST="${LITKIT_WORKSPACE}/corpus.manifest"

```

3.8.3 SLURM Resource Requests

Adjust for your cluster's hardware:

```

#SBATCH --partition=gpu           # Your GPU partition
#SBATCH --nodes=3                 # Total nodes (N-1 producers + 1 consumer)
#SBATCH --ntasks-per-node=1      # One task per node
#SBATCH --cpus-per-task=32       # CPUs for XML parsing
#SBATCH --mem=128G               # Memory per node
#SBATCH --gres=gpu:2             # GPUs per node
#SBATCH --time=10:00:00         # Wall time

```

3.9 Running Single-Node Builds

3.9.1 When to Use

- Corpus size < 50 GB
- Testing and development
- Quick validation runs

3.9.2 Basic Command

```
litkit --faiss-writer --build-only --rebuild --yes \  
      --tar-manifest /workspace/corpus.manifest
```

3.9.3 Using the Batch Script

```
# Edit vector_build_single.sbatch to set your paths  
sbatch -p YOUR_GPU_PARTITION vector_build_single.sbatch
```

3.9.4 Key Options

Option	Description
--faiss-writer	Enable FAISS index writes (required)
--build-only	Build without running a query
--rebuild	Wipe and rebuild from scratch
--update	Append new files only
--tar-manifest FILE	Path to manifest file
--tar-dir DIR	Directory containing tar files (alternative to manifest)

3.10 Running Multi-Node Builds

3.10.1 When to Use

- Corpus size > 50 GB
- Need faster builds via parallelism
- Multiple GPUs across multiple nodes

3.10.2 Architecture

With N nodes: - Nodes 0 to N-2: **Producers** (embed text, write segments) - Node N-1: **Consumer** (ingest segments, write FAISS indices)

3.10.3 Using the Batch Script

```
# Edit vector_build_multi.sbatch to set your paths  
# Adjust --nodes=N as needed  
  
sbatch -p YOUR_GPU_PARTITION vector_build_multi.sbatch
```

3.10.4 Bootstrap Phase

The script automatically creates empty FAISS indices before producers start:

```
litkit --faiss-writer --init-indices-only \  
      --papers-index hnsf --chunks-index flat
```

3.10.5 Monitoring Progress

```
# Watch job output  
tail -f litkit_multi_*.out  
  
# Check GPU utilization across nodes  
JOB=$(squeue --me -h -o %i | head -1)
```

```
for n in $(scontrol show hostnames $(squeue -j $JOB -h -o %N)); do
  srun --jobid=$JOB --overlap -N1 -n1 -w $n \
    nvidia-smi --query-gpu=utilization.gpu,memory.used --format=csv,noheader
done
```

3.11 Querying Your Index

3.11.1 Using a Question File

For long or complex questions, use `--question-file`:

```
# Create question file
cat > workspace/question.txt << 'EOF'
What molecular mechanisms have been proposed to explain
the interaction between protein X and protein Y in the
context of cellular signaling pathways?
EOF
```

```
# Query (retrieval + LLM)
litkit --question-file /workspace/question.txt
```

```
# Query (retrieval only, no LLM)
litkit --no-llm --question-file /workspace/question.txt
```

3.11.2 Interactive Queries

```
litkit "What is the role of autophagy in cancer?"
```

3.11.3 With External LLM API

```
litkit --llm-model gpt-4 \
  --openai-base-url "https://api.example.com/v1" \
  --openai-api-key "$API_KEY" \
  --question-file /workspace/question.txt
```

3.11.4 Key Query Options

Option	Description
<code>--question-file FILE</code>	Read question from file
<code>--no-llm</code>	Retrieval only (print context, skip LLM)
<code>--top-papers N</code>	Stage 1 shortlist size (default: 500)
<code>--top-chunks N</code>	Chunks for LLM context (default: 30)
<code>--per-paper-cap N</code>	Max chunks per paper (default: 3)
<code>--llm-model MODEL</code>	LLM model name

3.12 Troubleshooting

3.12.1 Verifying GPU Setup

Before running builds, verify PyTorch can see GPUs:

```
ch-run YOUR_IMAGE.sqfs \
  --cdi-dirs="$CDI_SPEC_DIR" --cdi=nvidia.com/gpu=all \
  --bind "$CUDA_LIBA:$CUDA_LIBA" \
  --bind "$CUDA_LIBB:$CUDA_LIBB" \
  --set-env="LD_LIBRARY_PATH=$CUDA_LIBA:$CUDA_LIBB" \
  -- /root/.local/share/uv/tools/litkit/bin/python - <<'PY'
import torch
print("Torch:", torch.__version__, "CUDA build:", torch.version.cuda)
print("CUDA available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("Device:", torch.cuda.get_device_name(0))
PY
```

Expected output:

```
Torch: 2.5.1 CUDA build: 12.5
CUDA available: True
Device: Tesla V100-PCIE-32GB
```

3.12.2 Monitoring SLURM Jobs

```
# Check job status
squeue --me

# Get job ID for completed job
jid=$(sacct -X -n --name=litkit-build --starttime=now-7days -o JobID | tail -1)

# View logs
less "litkit-build.$jid.out"
less "litkit-build.$jid.err"

# Get timing and status
sacct -X -j "$jid" -o JobID,JobName%30,Partition,State,ExitCode,Elapsed
```

3.12.3 “CUDA not available” inside container

1. Verify CDI spec exists:

```
ls $CDI_SPEC_DIR/nvidia.json
```

2. Check CDI flags in ch-run:

```
--cdi-dirs="$CDI_SPEC_DIR" --cdi=nvidia.com/gpu=all
```

3. Verify CUDA library bindings:

```
ls "$CUDA_LIBA"/libcuda*
ls "$CUDA_LIBB"/libcudart*
```

3.12.4 “database is locked” errors

These are transient on NFS/Lustre. Increase timeout:

```
export LITKIT_SQLITE_BUSY_TIMEOUT_MS=300000 # 5 minutes
```

3.12.5 Low GPU utilization (bursty pattern)

Usually caused by compressed .tar.gz files blocking parallel parsing.

Diagnosing: Check if multiple CPUs are parsing XML:

```
# In a separate terminal on the same node:
htop # Look for multiple Python threads with CPU usage
```

```
# Or check thread count:
ps -eLf | grep python | wc -l
```

```
# Watch overall CPU utilization:
watch -n1 'cat /proc/loadavg'
```

What success looks like: - Multiple CPU cores showing load during “Ingesting” phase - If only 1 core is busy → compressed tars blocking parallelism

Solutions: 1. Convert to uncompressed `.tar` files (see Why Uncompressed Tar Files?) 2. Increase `--parse-workers`: `bash litkit --faiss-writer --build-only --parse-workers 16 ... # default is 8` 3. Use more producer nodes

3.12.6 Producer failed, consumer waiting forever

Check producer logs for errors. If a producer crashes: 1. Fix the underlying issue 2. Restart the **entire** job (same node count!) 3. Checkpoints allow resumption from last successful point

3.12.7 Stale writer guard file

If a previous job crashed without cleanup:

```
rm workspace/.writer_guard
```

The batch script does this automatically at job start.

3.12.8 “pam_slurm_adopt” error when SSH to node

You don’t have an active allocation on that node:

```
# Check your allocations
squeue --me

# Request a new allocation
salloc -N1 -t 1:00:00 -p YOUR_PARTITION --no-shell
```

3.13 Command Reference

3.13.1 Environment Variables

Variable	Description
LITKIT_WORKSPACE	Output directory for indices/DB
LITKIT_TAR_DIR	Default tar directory
LITKIT_TAR_MANIFEST	Default manifest file
HF_HOME	HuggingFace model cache
HF_HUB_OFFLINE=1	Force offline mode (no downloads)
TRANSFORMERS_USE_SAFE_TENSORS=1	Use SafeTensors format
LITKIT_SQLITE_BUSY_TIMEOUT_MS	SQLite lock timeout (default: 120000)
LITKIT_SAVE_EVERY_SEC	FAISS save interval (default: 120)
LITKIT_DEBUG=1	Enable verbose debugging
LITKIT_QUIET=1	Suppress output

3.13.2 Key CLI Flags

```
# Build modes
--faiss-writer           # Enable FAISS writes (required for builds)
--embed-producer        # Producer mode (write segments, not FAISS)
--consume-only          # Consumer mode (ingest segments only)
--init-indices-only     # Create empty indices and exit

# Index types
--papers-index {hnsw,flat} # Paper index type (default: hnsw)
--chunks-index {ivfpq,flat} # Chunk index type (default: ivfpq)

# Multi-node
--shard-id N             # This producer's shard ID (0-indexed)
--num-shards N           # Total number of producer shards

# Performance
--parse-workers N        # Parallel XML parsing threads (default: 8)
--embed-devices SPEC     # GPU devices (auto, cpu, cuda:0,cuda:1)
--paper-embed-bs N       # Paper embedding batch size (default: 16)
--chunk-embed-bs N       # Chunk embedding batch size (default: 64)
```

3.13.3 Utility Scripts

Script	Description
setup_safetensors.sh	Convert HF models to SafeTensors format
vector_build_single.sbatch	SLURM script for single-node builds
vector_build_multi.sbatch	SLURM script for multi-node builds
vector_resume_consumer.sbatch	Resume consumer after producer completion